

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №3
«Модульне тестування програмного коду (Unit Testing)»
з дисципліни
«Основи програмної інженерії»

Виконала
ст. гр. ПЗП-25-4

Бондаренко Тетяна
Олександрівна

Прийняв:
В. І. Каук,
В.О. Гребенюк

Харків 2026

ЛАБОРАТОРНА РОБОТА 3

МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)

3.1 Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів- еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

1. Реалізація програмного модуля

На основі UML-моделі з лабораторної роботи №2

(<https://github.com/tetianabondarenko3-oss/bse-lr2-Bondarenko/tree/lab-02/lab-02>)

було реалізовано програмний модуль мовою програмування JavaScript.

У модулі реалізовано три основні класи:

- FactCard
- Category
- FavoriteList

Програмний модуль містить методи з нетривіальною логікою:

- умовні конструкції (if)
- цикли та обробку колекцій (find, filter, some)
- обробку виняткових ситуацій (throw new Error())

// КЛАС FactCard- картка з фактом (з діаграми класів ЛР 02)

```
class FactCard {  
  /**  
   * Конструктор картки факту  
   * @param {number} cardId - унікальний номер картки (ціле число > 0)  
   * @param {string} title - заголовок картки (не порожній рядок)  
   * @param {string} content - текст факту (не порожній рядок)  
   */  
  constructor(cardId, title, content) {  
    // Перевірка: cardId повинен бути цілим числом і більше 0  
    if (!Number.isInteger(cardId) || cardId <= 0) {  
      throw new Error("cardId must be a positive integer");  
    }  
    // Перевірка: title повинен бути рядком і не порожнім  
    if (typeof title !== "string" || title.trim() === "") {  
      throw new Error("title must be a non-empty string");  
    }  
  }  
}
```

```

    }
    // Перевірка: content повинен бути рядком і не порожнім
    if (typeof content !== "string" || content.trim() === "") {
        throw new Error("content must be a non-empty string");
    }

    this.cardId = cardId;
    this.title = title.trim();
    this.content = content.trim();
    this.status = "active"; // За замовчуванням картка активна
}

/**
 * display()- показати картку у вигляді рядка
 * Використовується при перегляді карток (FR-03)
 * @returns {string}
 */
display() {
    return `[${this.cardId}] ${this.title}: ${this.content}`;
}

/**
 * deactivate()- деактивувати картку (для модерації адміністратором)
 * Відповідає FR-07- модерування контенту
 * Кидає помилку, якщо картка вже неактивна
 */
deactivate() {
    if (this.status === "inactive") {
        throw new Error("Card is already inactive");
    }
    this.status = "inactive";
}
}

// КЛАС Category- категорія події (з діаграми класів ЛР 02)

class Category {
    /**
     * Конструктор категорії
     * @param {number} categoryId- унікальний номер категорії (ціле число > 0)
     * @param {string} name - назва категорії (не порожній рядок)
     */
    constructor(categoryId, name) {
        if (!Number.isInteger(categoryId) || categoryId <= 0) {
            throw new Error("categoryId must be a positive integer");
        }
    }
}

```

```

    }
    if (typeof name !== "string" || name.trim() === "") {
        throw new Error("name must be a non-empty string");
    }

    this.categoryId = categoryId;
    this.name = name.trim();
    this.cards = []; // Масив карток у цій категорії
}

/**
 * addCard()- додати картку до категорії
 * @param {FactCard} card- об'єкт типу FactCard
 */
addCard(card) {
    // Перевірка: card повинен бути екземпляром класу FactCard
    if (!(card instanceof FactCard)) {
        throw new Error("card must be an instance of FactCard");
    }
    // Перевірка: картка з таким id вже є в категорії
    const exists = this.cards.find((c) => c.cardId === card.cardId);
    if (exists) {
        throw new Error(`Card with id ${card.cardId} already exists in category`);
    }
    this.cards.push(card);
}

/**
 * getActiveCards()- повернути лише активні картки
 * @returns {FactCard[]}
 */
getActiveCards() {
    return this.cards.filter((c) => c.status === "active");
}

/**
 * selectCategory()- вибрати категорію та дізнатись скільки активних карток
 * Відповідає FR-01- вибір категорії події
 * @returns {string}
 */
selectCategory() {
    const count = this.getActiveCards().length;
    if (count === 0) {
        return `Category "${this.name}" has no active cards`;
    }
}

```

```

    }
    return `Category "${this.name}" selected. Active cards: ${count}`;
  }
}

```

// КЛАС FavoriteList- список обраного

```

class FavoriteList {
  /**
   * Конструктор списку обраного
   * @param {number} listId- унікальний номер списку (ціле число > 0)
   */
  constructor(listId) {
    if (!Number.isInteger(listId) || listId <= 0) {
      throw new Error("listId must be a positive integer");
    }
    this.listId = listId;
    this.cards = [];
    this.createdAt = new Date();
  }

  /**
   * addCard()- додати картку в обране
   * Відповідає FR-05- збереження в «Обране»
   * @param {FactCard} card
   */
  addCard(card) {
    if (!(card instanceof FactCard)) {
      throw new Error("card must be an instance of FactCard");
    }
    if (this.cards.find((c) => c.cardId === card.cardId)) {
      throw new Error("Card already in favorites");
    }
    this.cards.push(card);
  }

  /**
   * removeCard()- видалити картку з обраного за id
   * @param {number} cardId
   */
  removeCard(cardId) {
    if (!Number.isInteger(cardId) || cardId <= 0) {
      throw new Error("cardId must be a positive integer");
    }
    const index = this.cards.findIndex((c) => c.cardId === cardId);
  }
}

```

```

    if (index === -1) {
        throw new Error(`Card with id ${cardId} not found in favorites`);
    }
    this.cards.splice(index, 1);
}

/**
 * hasCard()- перевірити, чи є картка в обраному
 * @param {number} cardId
 * @returns {boolean}
 */
hasCard(cardId) {
    return this.cards.some((c) => c.cardId === cardId);
}

// Експортуємо класи для використання у тестах
module.exports = { FactCard, Category, FavoriteList };

```

2. Проєктування тестів (ЕР та BVA)

Для кожного методу були визначені:

- класи еквівалентності (ЕР- Equivalence Partitioning)
- граничні значення (BVA- Boundary Value Analysis)

Результати оформлені у вигляді **таблиці тест-кейсів**.

Тест-кейс	Вхідні дані / Опис	Очікуваний результат	Техніка	Статус
ТС-01	FactCard: коректне створення картки	Усі поля заповнені правильно	ЕР (Допустимий клас)	pass
ТС-02	FactCard: cardId = 0	Кидає помилку	BVA (Межа 0)	pass
ТС-03	FactCard: від'ємний cardId = -1	Кидає помилку	ЕР (Від'ємні числа)	pass
ТС-04	FactCard: порожній title	Кидає помилку	ЕР (Порожній рядок)	pass
ТС-05	FactCard: порожній content	Кидає помилку	ЕР (Порожній рядок)	pass
ТС-06	FactCard.display()	Повертає відформатований рядок	Позитивний тест	pass
ТС-07	FactCard.deactivate()	Статус картки стає inactive	Позитивний тест	pass

TC-08	FactCard.deactivate() : повторно	Кидає помилку	ЕР (Недопустимий стан)	pass
TC-09	Category: коректне створення	Об'єкт створено успішно	ЕР (Допустимий клас)	pass
TC-10	Category: порожня назва	Кидає помилку	ЕР (Порожній рядок)	pass
TC-11	Category.addCard(): одна картка	Картка додана в масив	Позитивний тест	pass
TC-12	Category.addCard(): той самий id	Кидає помилку	ЕР (Дублікат)	pass
TC-13	Category.addCard(): не-FactCard об'єкт	Кидає помилку	ЕР (Недопустимий тип)	pass
TC-14	Category.getActiveCards()	Повертає лише активні картки	Позитивний + ЕР	pass
TC-15	Category.selectCategory(): 0 карток	Рядок для нижньої межі	BVA (0 карток)	pass
TC-16	Category.selectCategory(): 1 картка	Рядок для мінімуму	BVA (1 картка)	pass
TC-17	Category.selectCategory(): кілька карток	Рядок для списку	ЕР (Більше 1 картки)	pass
TC-18	FavoriteList: коректне створення	Об'єкт створено	ЕР (Допустимий ID)	pass
TC-19	FavoriteList: listId = 0	Кидає помилку	BVA (Межа 0)	pass
TC-20	FavoriteList.addCard(): успіх	Картка додана в обране	Позитивний тест	pass
TC-21	FavoriteList.addCard(): дублікат	Кидає помилку	ЕР (Вже в обраному)	pass
TC-22	FavoriteList.removeCard(): успіх	Картка видалена з обраного	Позитивний тест	pass
TC-23	FavoriteList.removeCard(): немає id	Кидає помилку	ЕР (Відсутній ID)	pass

3. Реалізація модульних тестів

Для тестування було використано фреймворк Jest.

```
PS E:\универ\опи\bse-lr3-Bondarenko> Set-ExecutionPolicy Bypass -Scope Process
PS E:\универ\опи\bse-lr3-Bondarenko> npm install
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 267 packages, and audited 268 packages in 12s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS E:\универ\опи\bse-lr3-Bondarenko>
```

```
PS E:\универ\опи\bse-lr3-Bondarenko> git remote add origin https://github.com/tetianabondarenko3-oss/
error: remote origin already exists.
PS E:\универ\опи\bse-lr3-Bondarenko> node -v
v24.14.1
PS E:\универ\опи\bse-lr3-Bondarenko> npm -v
11.11.0
PS E:\универ\опи\bse-lr3-Bondarenko>
```

Усі тести були написані відповідно до патерну AAA:

- Arrange- підготовка даних
- Act- виконання дії
- Assert- перевірка результату

/**

* *Тести для модуля Touch Up Smart*

* *ЛР 3- Модульне тестування*

* *Студент: Бондаренко Тетяна Олександрівна, ПЗПІ-25-4*

*

* *Фреймворк: Jest*

* *Техніки: EP (Equivalence Partitioning- еквівалентне розбиття)*

* *BVA (Boundary Value Analysis- аналіз граничних значень)*

* *Патерн кожного тесту: AAA (Arrange → Act → Assert)*

*/

```
const { FactCard, Category, FavoriteList } = require("../src/touchUpSmart");
```

// БЛОК 1: Клас FactCard- перевірка конструктора

```
test("TC-01 | FactCard: коректне створення картки- всі поля заповнені правильно (EP: допустимий клас)", () => {
```

```
  // Arrange- готуємо вхідні дані
```

```
  const cardId = 1;
```

```
  const title = "Цікавий факт про тварин";
```

```
  const content = "Восьминоги мають три серця";
```



```

// Act- викликаємо дію
const card = new FactCard(cardId, title, content);

// Assert- перевіряємо результат
expect(card.cardId).toBe(1);
expect(card.title).toBe("Цікавий факт про тварин");
expect(card.content).toBe("Восьминоги мають три серця");
expect(card.status).toBe("active");
});

test("TC-02 | FactCard: cardId = 0 повинен кидати помилку (BVA: значення 0, межа між недопустимим і допустимим)", () => {
  // Arrange- недопустиме граничне значення
  const cardId = 0;

  // Act + Assert- перевіряємо що кидается помилка
  expect(() => new FactCard(cardId, "title", "content")).toThrow(
    "cardId must be a positive integer"
  );
});

test("TC-03 | FactCard: від'ємний cardId = -1 повинен кидати помилку (EP: клас від'ємних чисел)", () => {
  // Arrange
  const cardId = -1;

  // Act + Assert
  expect(() => new FactCard(cardId, "title", "content")).toThrow(
    "cardId must be a positive integer"
  );
});

test("TC-04 | FactCard: порожній title повинен кидати помилку (EP: клас порожніх рядків)", () => {
  // Arrange
  const title = "";

  // Act + Assert
  expect(() => new FactCard(1, title, "content")).toThrow(
    "title must be a non-empty string"
  );
});

test("TC-05 | FactCard: порожній content повинен кидати помилку (EP: клас порожніх рядків)", () => {

```

```

// Arrange
const content = "";

// Act + Assert
expect(() => new FactCard(1, "title", content)).toThrow(
  "content must be a non-empty string"
);
});

// БЛОК 2: Клас FactCard- методи display() та deactivate()

test("TC-06 | FactCard.display(): повертає коректно відформатований рядок (позитивний)", () => {
  // Arrange
  const card = new FactCard(1, "Факт дня", "Мед ніколи не псується");

  // Act
  const result = card.display();

  // Assert
  expect(result).toBe("[1] Факт дня: Мед ніколи не псується");
});

test("TC-07 | FactCard.deactivate(): статус картки стає inactive (позитивний)", () => {
  // Arrange
  const card = new FactCard(1, "Факт", "Текст факту");

  // Act
  card.deactivate();

  // Assert
  expect(card.status).toBe("inactive");
});

test("TC-08 | FactCard.deactivate(): повторна деактивація кидає помилку (ЕР: недопустимий стан)", () => {
  // Arrange- картка вже деактивована
  const card = new FactCard(1, "Факт", "Текст факту");
  card.deactivate();

  // Act + Assert- друга деактивація повинна кинути помилку
  expect(() => card.deactivate()).toThrow("Card is already inactive");
});

```

// БЛОК 3: Клас Category- конструктор та метод addCard()

```
test("TC-09 | Category: коректне створення категорії (ЕР: допустимий клас)", () => {  
  // Arrange + Act  
  const cat = new Category(1, "Свято");  
  
  // Assert  
  expect(cat.categoryId).toBe(1);  
  expect(cat.name).toBe("Свято");  
  expect(cat.cards).toHaveLength(0);  
});
```

```
test("TC-10 | Category: порожня назва категорії кидає помилку (ЕР: клас порожніх рядків)", () => {  
  // Arrange  
  const name = "";  
  
  // Act + Assert  
  expect(() => new Category(1, name)).toThrow("name must be a non-empty string");  
});
```

```
test("TC-11 | Category.addCard(): додавання однієї картки (позитивний)", () => {  
  // Arrange  
  const cat = new Category(1, "Бізнес");  
  const card = new FactCard(1, "Порада для бізнесу", "Слухайте більше ніж говоріть");  
  
  // Act  
  cat.addCard(card);  
  
  // Assert  
  expect(cat.cards).toHaveLength(1);  
});
```

```
test("TC-12 | Category.addCard(): додавання картки з тим самим id кидає помилку (ЕР: дублікат)", () => {  
  // Arrange- картка вже є в категорії  
  const cat = new Category(1, "Бізнес");  
  const card = new FactCard(1, "Порада", "Текст");  
  cat.addCard(card);  
  
  // Act + Assert
```

```
expect() => cat.addCard(card)).toThrow("already exists in category");
});
```

```
test("TC-13 | Category.addCard(): передача не-FactCard об'єкту кидає помилку  
(ЕР: недопустимий тип)", () => {
```

```
  // Arrange- передаємо звичайний об'єкт замість FactCard
```

```
  const cat = new Category(1, "Спорт");
```

```
  const fakeCard = { cardId: 1, title: "fake" };
```

```
  // Act + Assert
```

```
  expect() => cat.addCard(fakeCard)).toThrow(  
    "card must be an instance of FactCard"
```

```
  );
```

```
});
```

```
// БЛОК 4: Клас Category- методи getActiveCards() та selectCategory()
```

```
test("TC-14 | Category.getActiveCards(): повертає лише активні картки  
(позитивний + ЕР)", () => {
```

```
  // Arrange- 2 картки: 1 активна, 1 деактивована
```

```
  const cat = new Category(1, "Свято");
```

```
  const card1 = new FactCard(1, "Активна картка", "Текст 1");
```

```
  const card2 = new FactCard(2, "Неактивна картка", "Текст 2");
```

```
  card2.deactivate(); // деактивуємо другу картку
```

```
  cat.addCard(card1);
```

```
  cat.addCard(card2);
```

```
  // Act
```

```
  const activeCards = cat.getActiveCards();
```

```
  // Assert- повинна бути лише 1 активна
```

```
  expect(activeCards).toHaveLength(1);
```

```
  expect(activeCards[0].cardId).toBe(1);
```

```
});
```

```
test("TC-15 | Category.selectCategory(): категорія без активних карток (BVA: 0  
карток- нижня межа)", () => {
```

```
  // Arrange- порожня категорія
```

```
  const cat = new Category(1, "Тест");
```

```
  // Act
```

```
  const result = cat.selectCategory();
```

```
  // Assert
```

```
  expect(result).toBe('Category "Тест" has no active cards');
```

```
});
```

```
test("TC-16 | Category.selectCategory(): рівно 1 активна картка (BVA: мінімум 1 картка)", () => {  
  // Arrange  
  const cat = new Category(1, "Побачення");  
  cat.addCard(new FactCard(1, "Тема для розмови", "Розкажи про своє хобі"));  
  
  // Act  
  const result = cat.selectCategory();  
  
  // Assert  
  expect(result).toContain("Active cards: 1");  
  expect(result).toContain("Побачення");  
});
```

```
test("TC-17 | Category.selectCategory(): кілька активних карток (EP: більше 1 картки)", () => {  
  // Arrange  
  const cat = new Category(1, "Вечірка");  
  cat.addCard(new FactCard(1, "Факт 1", "Текст 1"));  
  cat.addCard(new FactCard(2, "Факт 2", "Текст 2"));  
  cat.addCard(new FactCard(3, "Факт 3", "Текст 3"));  
  
  // Act  
  const result = cat.selectCategory();  
  
  // Assert  
  expect(result).toContain("Active cards: 3");  
});
```

*// БЛОК 5: Клас **FavoriteList**- конструктор*

```
test("TC-18 | FavoriteList: коректне створення (EP: допустимий listId)", () => {  
  // Arrange + Act  
  const list = new FavoriteList(1);  
  
  // Assert  
  expect(list.listId).toBe(1);  
  expect(list.cards).toHaveLength(0);  
  expect(list.createdAt).toBeInstanceOf(Date);  
});
```

```
test("TC-19 | FavoriteList: listId = 0 кидає помилку (BVA: значення 0, межа між недопустимим і допустимим)", () => {
```

```
// Arrange  
const listId = 0;
```

```
// Act + Assert  
expect((() => new FavoriteList(listId)).toThrow(  
  "listId must be a positive integer"  
));  
});
```

// БЛОК 6: Клас FavoriteList- методи addCard(), removeCard(), hasCard()

```
test("TC-20 | FavoriteList.addCard(): успішне додавання картки в обране  
(позитивний)", () => {
```

```
  // Arrange  
  const list = new FavoriteList(1);  
  const card = new FactCard(5, "Улюблений факт", "Текст улюбленого факту");
```

```
  // Act  
  list.addCard(card);
```

```
  // Assert  
  expect(list.cards).toHaveLength(1);  
  expect(list.hasCard(5)).toBe(true);  
});
```

```
test("TC-21 | FavoriteList.addCard(): додавання дубліката кидає помилку (ЕР:  
картка вже в обраному)", () => {
```

```
  // Arrange  
  const list = new FavoriteList(1);  
  const card = new FactCard(1, "Факт", "Текст");  
  list.addCard(card);
```

```
  // Act + Assert  
  expect((() => list.addCard(card)).toThrow("Card already in favorites"));  
});
```

```
test("TC-22 | FavoriteList.removeCard(): успішне видалення картки з обраного  
(позитивний)", () => {
```

```
  // Arrange- картка є в обраному  
  const list = new FavoriteList(1);  
  const card = new FactCard(1, "Факт", "Текст");  
  list.addCard(card);
```

```
  // Act  
  list.removeCard(1);
```

```
// Assert
expect(list.cards).toHaveLength(0);
expect(list.hasCard(1)).toBe(false);
});

test("TC-23 | FavoriteList.removeCard(): видалення картки якої немає кидає помилку (EP: відсутній id)", () => {
  // Arrange- порожній список
  const list = new FavoriteList(1);

  // Act + Assert
  expect(() => list.removeCard(99)).toThrow("not found in favorites");
});
```

Результати запуску тестів:

Було виконано 23 тест-кейси.

Усі тести завершилися успішно.

```
PS E:\универ\опи\bse-lr3-Bondarenko> npm test

> bse-lr3-bondarenko@1.0.0 test
> jest --coverage

PASS tests/touchUpSmart.test.js
  ✓ TC-01 | FactCard: коректне створення картки – всі поля заповнені правильно (EP: допустимий клас) (1 ms)
  ✓ TC-02 | FactCard: cardId = 0 повинен кидати помилку (BVA: значення 0, межа між недопустимим і допустимим) (6 ms)
  ✓ TC-03 | FactCard: від'ємний cardId = -1 повинен кидати помилку (EP: клас від'ємних чисел)
  ✓ TC-04 | FactCard: порожній title повинен кидати помилку (EP: клас порожніх рядків)
  ✓ TC-05 | FactCard: порожній content повинен кидати помилку (EP: клас порожніх рядків)
  ✓ TC-06 | FactCard.display(): повертає коректно відформатований рядок (позитивний)
  ✓ TC-07 | FactCard.deactivate(): статус картки стає inactive (позитивний)
  ✓ TC-08 | FactCard.deactivate(): повторна деактивація кидає помилку (EP: недопустимий стан) (1 ms)
  ✓ TC-09 | Category: коректне створення категорії (EP: допустимий клас)
  ✓ TC-10 | Category: порожня назва категорії кидає помилку (EP: клас порожніх рядків) (1 ms)
  ✓ TC-11 | Category.addCard(): додавання однієї картки (позитивний)
  ✓ TC-12 | Category.addCard(): додавання картки з тим самим id кидає помилку (EP: дублікат) (1 ms)
  ✓ TC-13 | Category.addCard(): передача не-FactCard об'єкту кидає помилку (EP: недопустимий тип)
  ✓ TC-14 | Category.getActiveCards(): повертає лише активні картки (позитивний + EP)
  ✓ TC-15 | Category.selectCategory(): категорія без активних карток (BVA: 0 карток – нижня межа)
  ✓ TC-16 | Category.selectCategory(): рівно 1 активна картка (BVA: мінімум 1 картка)
  ✓ TC-17 | Category.selectCategory(): кілька активних карток (EP: більше 1 картки)
  ✓ TC-18 | FavoriteList: коректне створення (EP: допустимий listId)
  ✓ TC-19 | FavoriteList: listId = 0 кидає помилку (BVA: значення 0, межа між недопустимим і допустимим)
  ✓ TC-20 | FavoriteList.addCard(): успішне додавання картки в обране (позитивний)
  ✓ TC-21 | FavoriteList.addCard(): додавання дублікату кидає помилку (EP: картка вже в обраному)
  ✓ TC-22 | FavoriteList.removeCard(): успішне видалення картки з обраного (позитивний) (1 ms)
  ✓ TC-23 | FavoriteList.removeCard(): видалення картки якої немає кидає помилку (EP: відсутній id)

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 94.54   | 92.85    | 100     | 94      |
touchUpSmart.js | 94.54   | 92.85    | 100     | 94      | 73,147,161
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       23 passed, 23 total
Snapshots:   0 total
Time:        0.48 s
Ran all test suites.
```

4. Аналіз покриття коду

Після запуску тестів було автоматично згенеровано coverage-звіт.

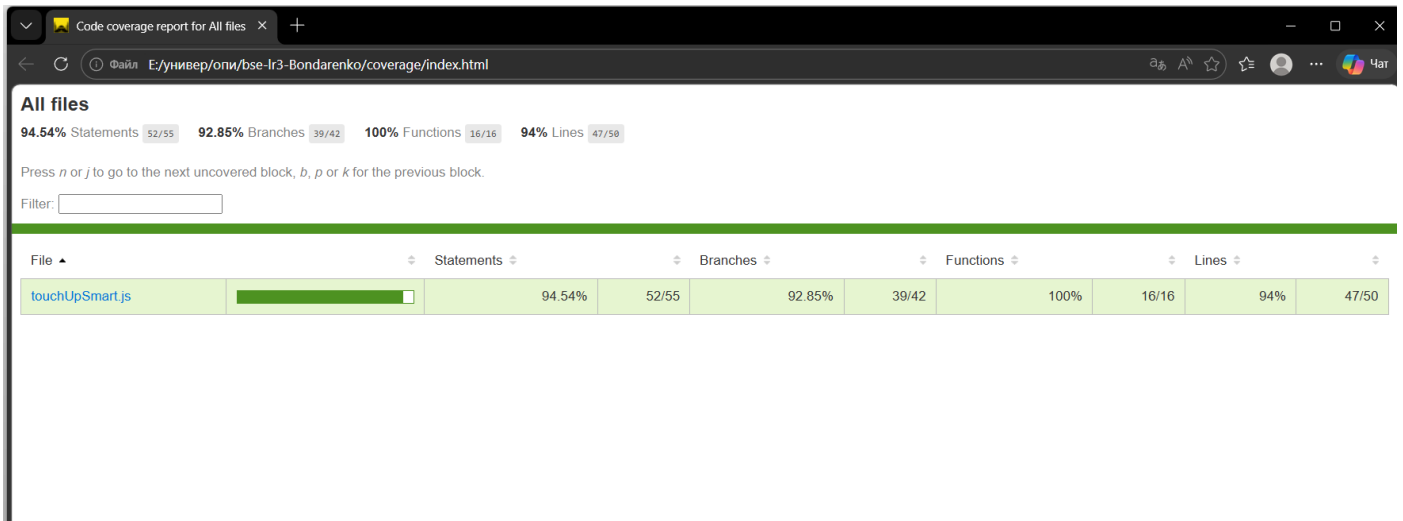
Отримані результати:

Statements: 94.54%

Branches: 92.85%

Functions: 100%

Lines: 94%



```
120 }
121 }
122 // =====
123 // Клас FavoriteList - список обраного (з діаграми класів ЛР 02)
124 // =====
125
126 class FavoriteList {
127     /**
128      * Конструктор списку обраного
129      * @param {number} listId - унікальний номер списку (ціле число > 0)
130      */
131     constructor(listId) {
132         6x if (!Number.isInteger(listId) || listId <= 0) {
133             1x throw new Error("listId must be a positive integer");
134         }
135         5x this.listId = listId;
136         5x this.cards = [];
137         5x this.createdAt = new Date();
138     }
139
140     /**
141      * addCard() - додати картку в обране
142      * Відповідає FR-05 - збереження в "Обране"
143      * @param {FactCard} card
144      */
145     addCard(card) {
146         4x if (!(card instanceof FactCard)) {
147             throw new Error("card must be an instance of FactCard");
148         }
149         4x if (this.cards.find((c) => c.cardId === card.cardId)) {
150             1x throw new Error("Card already in favorites");
151         }
152         3x this.cards.push(card);
153     }
154
155     /**
156      * removeCard() - видалити картку з обраного за id
157      * @param {number} cardId
158      */
159     removeCard(cardId) {
160         2x if (!Number.isInteger(cardId) || cardId <= 0) {
161             throw new Error("cardId must be a positive integer");
162         }
163         2x const index = this.cards.findIndex((c) => c.cardId === cardId);
164         2x if (index === -1) {
165             1x throw new Error("Card with id ${cardId} not found in favorites");
166         }
167         1x this.cards.splice(index, 1);
168     }
169
170     /**
171      * hasCard() - перевірити, чи є картка в обраному
172      * @param {number} cardId
173      * @returns {boolean}
174      */
175     hasCard(cardId) {
```

Отриманий показник line coverage = 94%, що перевищує мінімально необхідний поріг 80%.

Аналіз результатів

Було досягнуто високого рівня покриття коду завдяки:

- позитивним тестам
- негативним тестам
- перевірці граничних значень
- перевірці виняткових ситуацій

Непокритими залишилися лише окремі рядки коду, які не впливають на основну логіку роботи модуля.

5. GitHub-репозиторій

Для збереження коду та тестів було використано GitHub-репозиторій.

У репозиторій було завантажено:

- програмний модуль
- тестовий набір
- конфігурацію Jest
- coverage-звіт
- GitHub Actions workflow

Посилання на репозиторій: <https://github.com/tetianabondarenko3-oss/bse-lr3-Bondarenko.git>

6. GitHub Actions (бонусне завдання)

Було налаштовано автоматичний запуск тестів через GitHub Actions.

Після кожного git push система автоматично:

- встановлює залежності
- запускає Jest
- перевіряє проходження тестів

The screenshot displays the GitHub Actions interface for the repository `tetianabondarenko3-oss / bse-lr3-Bondarenko`. The top navigation bar includes links for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The left sidebar shows the 'Actions' tab with a 'New workflow' button and a list of workflow runs. The main content area shows 'All workflows' with a search bar and a table of workflow runs. The table lists two workflow runs: 'docs: add latest fixes to code and comments' and 'feat: complete unit tests and coverage report for lr3'. The 'feat: complete unit tests and coverage report for lr3' workflow run is selected, showing a summary of the job. The summary includes the status 'Success', duration '17s', and a list of tests (test) with their status 'Success' and duration '12s'. The interface also includes a sidebar with navigation links and a bottom section for annotations.

ВИСНОВКИ

У ході лабораторної роботи було виконано модульне тестування трьох класів проєкту «Touch Up Smart»: FactCard, Category та FavoriteList. Для створення тест-кейсів було застосовано техніки еквівалентного розбиття (EP) та аналізу граничних значень (BVA).

Було написано та успішно запущено 23 тест-кейси, які повністю покривають логіку роботи програми як для валідних, так і для некоректних вхідних даних (ексепшні). За допомогою фреймворку Jest було досягнуто показник покриття рядків коду (Line Coverage = 94%), що повністю задовольняє та перевищує встановлений критерій у 80%. Непокритими залишилися поодинокі рядки перевірок додаткових умов (рядки 73, 147, 161), які не впливають на загальну стабільність системи. Додатково було успішно інтегровано інструмент автоматизації тестування GitHub Actions (CI), що підтверджується зеленими статусами збірок у репозиторії.